

CYBERSECURITY IN API DEVELOPMENT

Erik Markula
Bachelor's Thesis
Spring 2026
Degree Programme in Information Technology
Oulu University of Applied Sciences

ABSTRACT

Oulu University of Applied Sciences
Degree Programme in Information Technology

Author(s): Erik Markula
Title of thesis: Cybersecurity in API Development
Term and year of completion: Spring 2026
Pages: 35

The objective of this thesis was to create an API for Mission Grey Inc. and examine methods for improving the security of internet-facing APIs, with a focus on commonly occurring vulnerabilities and practical mitigation strategies.

The theoretical background was based on established API security principles and the OWASP API Security Top 10, which provides a structured overview of the most critical risks affecting APIs. A practical approach was taken to demonstrate implementation details, and research was conducted mainly through literature study. Django Rest Framework was used to demonstrate how security controls can be applied in practice.

The outcome of the thesis is a set of methods and practices that can be used to improve the security of any API implementation. API for Mission Grey Inc. was created. These methods serve as a foundation for further development of a well secured API for web application use.

CONTENTS

ABSTRACT	2
CONTENTS.....	3
GLOSSARY.....	5
1 INTRODUCTION	7
2 CONNECTIONS IN WEB APPLICATIONS	8
2.1 API	8
2.2 API as a product.....	9
2.3 API management.....	10
3 API SECURITY.....	12
3.1 OWASP	12
3.2 Broken Object Level Authorization	12
3.3 Broken Authentication	14
3.4 Broken Object Property Level Authorization.....	16
3.5 Broken function level authorization.....	16
3.6 Unrestricted access to sensitive business flows.....	17
3.7 Unrestricted Resource Consumption.....	18
3.8 Source Code Analysis Tools	20
4 RISK MANAGEMENT IN WEB APPLICATIONS.....	21
4.1 API Governance	21
4.1.1 Regulation	22
4.1.2 Assessment.....	22
4.1.3 Management	23
4.2 Architecture	24
4.2.1 API gateway	24
4.2.2 Network Topology.....	26
5 INCIDENT RESPONSE.....	27
5.1 Observability.....	27
5.2 Response	28
6 FUTURE DEVELOPMENT AND DISCUSSION	30
6.1 Discussion	30
6.2 Future development	31

REFERENCES..... 32

GLOSSARY

AI	Artificial Intelligence
API	Application Programming Interface
BOLA	Broken Object Level Authorization
BOPLA	Broken object property-level authorization
CD	Continuous Deployment
CI	Continuous Integration
CSRF	Cross-Site Request Forgery
CSIRT	Computer Security Incident Response Team
DDoS	Distributed denial of service
DoS	Denial of Service
HTTP	HyperText Transfer Protocol
IDE	Integrated Development Environment
IP	Internet Protocol
JSON	JavaScript Object Notation
ORM	Object-Relational Mapping
OWASP	Open Worldwide Application Security Project
REST	Representational State Transfer
SAST	Source Code Analysis Tool
SSH	Secure Shell Protocol
SQL	Simple Query Language

URL	Uniform Resource Locator
UX	User Experience

1 INTRODUCTION

The client of this thesis is Mission Grey Incorporated, which offers an intelligence platform utilizing custom data models and artificial intelligence. The company operates on three different continents and employs fifteen people.

In an age where building services is easier and faster than ever with the aid of LLM models and AI tools, managing the security of a service through human intervention and careful security considerations is more important than ever. As modern API-based systems continue to grow in complexity and exposure, the attack surface of services expand significantly. These systems are an attractive target for abuse and automated attacks.

As such, the purpose of this thesis is to explore API concepts and ways to improve the security of a service where network traffic is received from the public internet, with code examples specifically in the context of Django Rest Framework. API security is approached from both a technical and organizational perspective, addressing implementation, architectural, and governance considerations. Other types of services such as frontend clients have their own set of requirements for security, and the contents of this thesis cannot be directly applied to those other types of services.

By discussing these strategies, the work aims to provide practical guidance for improving the security posture of API-driven services in modern, internet-facing environments.

2 CONNECTIONS IN WEB APPLICATIONS

To create a system comprised of multiple software or hardware components that might be in completely different locations and networks, those components need to be connected. Communicating between computers and devices, is often implemented using an Application Programming Interface, which is the topic of this chapter.

2.1 API

An API is a set of rules that enables a software program to transmit data to another software program, usually through the internet. Accounting for at least 57% of all internet traffic (CloudFlare 2024), APIs are one of the most important concepts in modern web application design, working as abstraction layers between systems and making it possible to send data securely, start computing processes, or re-use existing functionalities and systems. These features enable flexible separation of concerns between systems from an architectural viewpoint and make architecture styles such as microservices and serverless possible. (Cloudflare s.a. a);(Dynowski & Dulak 2025, chapter 1.)

APIs that connect through the internet usually exchange data through the HTTP protocol, that consists of a request and a response, with request being sent by a client to trigger an action on a server, and response being the answer that the server sends in response to a request. (Mozilla 2025.)

There are many kinds of APIs and in the context of network-based APIs they are usually categorized by access type. Private APIs, also known as internal APIs, are developed internally within an organization and intended to be used to exchange data among organizational units. Public APIs are exposed to the public internet and services or data provided by public APIs can be free or monetized by the owning organization. Partner APIs are accessible only by the organization developing them or the organization's partners. (Dynowski & Dulak 2025, chapter 1.)

2.2 API as a product

APIs can be often treated as only an integration technology that connects services, however in some cases APIs may be treated as a product. Generating revenues from APIs is called API monetization. There are several monetization models that support different kinds of business models and might use various kinds of mechanisms for securing the APIs and identifying paying customers (Table 1). (Pedro 2024, chapter 3.)

Anyone can access an API on the public internet if they have and URL pointing to it. To prevent users from accessing an API for free or without limits, access tokens or other credentials like API keys are utilized. These credentials are used to prove that a user has access to the API and that they have permissions to access certain resources. Because credentials can be stolen or used wrongly, other kinds of security methods are relevant and credentials only work as the first line of defence. (Haro 2025, chapter 7.1.)

TABLE 1. Types of API monetization models (Sandoval 2025)

<i>Model</i>	<i>Description</i>
Pay-per-use	A user pays for utilizing the API, measured in the number of calls, the amount of data exchanged, or some other usage metric.
Subscription	A user pays a fixed subscription fee, usually monthly or annually.
Freemium	A user can use some features of the API for free, but needs to pay for advanced features, higher usage, or premium support.

Transaction Fee	A user pays a fee for every transaction made via the API.
Revenue sharing	An organization collects a set amount of revenue when their API is used by a third-party in their own product.

Treating an API like a product means treating it with the same attention and care as any other company asset. The goal of this approach is to create a positive user experience and provide up-to-date documentation, usage guides, and user support, making the API accessible and standardized. Important aspects to consider are maintaining the API for extended periods of time, following good practices such as versioning and release or deprecation procedures, and hardening the security of the API. (Dynowski & Dulak 2025, chapter 1);(Pedro 2024, chapter 3.)

2.3 API management

API management is the practice of creating a standardized process around each step of the development process. Vulnerabilities might be born from mismanaging APIs, as without proper management an API might still be in an active state and open to access without anyone knowing about it. These are called shadow APIs and are often undocumented. Shadow APIs are not maintained and can become potential security risks, leading to unauthorized access, data breaches, or API abuse.

Small organizations with only a few APIs can manage with little API management, however on a larger scale and with multiple teams operating different APIs, the importance of proper API management becomes apparent. For that purpose, there exists solutions called API platforms, which are a suite of tools to allows API teams to develop, publish, manage, and consume APIs. These are usually

commercial products provided by companies, common API platforms include Amazon API Gateway, Azure API management, and Google Cloud Apigee.

Proper API management and usage of API management platforms increase security and ease development, as they provide reusable components that would otherwise take time to develop, and allows centralized control to all traffic going through the management platform components. (Dynowski & Dulak 2025, chapter 1.);(Pedro 2024, chapter 3.);(Gallagher, 2025.)

3 API SECURITY

Anything connected to the internet can be attacked and should be protected against malicious intent and threat actors. The practice of designing, building, and operating APIs securely to detect and prevent vulnerabilities is called API security. Most important parts of an application that affect the security have a part to play in authentication, authorization, user input, data validation, network configuration, and observability. In the age of autonomous vehicles and smart cities, security and especially API security is growing more important. (Haro 2025, chapter 1.);(China & Goodwin);(OWASP s.a. a.)

3.1 OWASP

OWASP is a nonprofit foundation with tens of thousands of members that works to improve the security of software. They maintain and create many open-source projects, standards, and documentation related to software security. For increasing awareness and improving standards of software security, they provide info on useful ways to create secure code and development practices. OWASP provides a list of the most common security vulnerabilities that can be used to as a reference to make software more resilient against attacks and in improving security. (OWASP 2026)

Common security vulnerabilities

In this section common vulnerabilities according to OWASP and the ways of mitigating them in the context of APIs, and especially in Django and Django Rest Framework are explored. These are all widespread problems in APIs and are used by threat actors in many kinds of attacks, which will be discussed later.

3.2 Broken Object Level Authorization

Object Level Authorization is an access control mechanism that when broken, allows a user of an API to have access to resources that the user should not have

access to. This is extremely common in API-based applications because a client's state is not tracked and parameters sent with the URL like object IDs are used to decide which objects to access. (OWASP 2023)

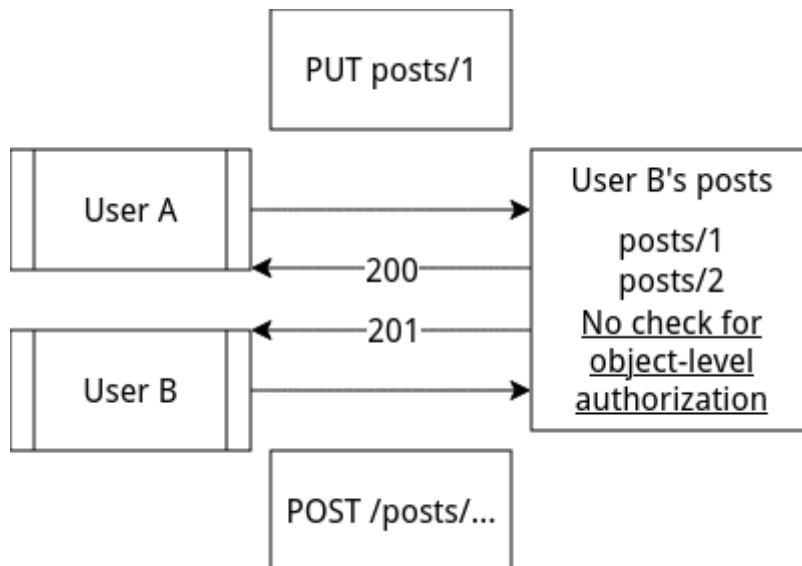


FIGURE 1. User A can access user B's posts by their ID. No knowledge of the ID is needed, by making requests with random IDs any resource of user B could be accessed.

A user can try to access random resources with different IDs by modifying the URL and potentially using, editing, or deleting data that belongs to another user if successful (Figure 1). (Haro 2025, chapter 4.2);

Code begins.

```

class IsOwner(permissions.BasePermission):
    """
    Assumes the model instance has an `user` attribute.
    """
    def has_object_permission(self, request, view, obj):
        # Instance must have an attribute named `user`.
        return obj.user == request.user

class ExampleViewSet(CreateListRetrieveViewSet):
    permission_classes = [custom_permissions.IsOwner]
  
```

Code ends.

FIGURE 2. Defining and using permissions in Django Rest Framework (Django REST framework s.a. a)

In Django Rest Framework BOLA can be addressed by creating a permission and applying it for every request the API receives. The check for permissions is made before any other code is executed, rejecting requests made by users whose identification such as an ID does not match the owner of the resource (Figure 2).

3.3 Broken Authentication

An API usually has some mechanism for authentication, which is used to identify users which resources users should have access to. Incorrectly implemented authentication allows attackers to compromise other user's identities temporarily or permanently. An authentication mechanism can be considered broken when it is designed to permit weak passwords, allow brute forcing, or when allowing the sending of credentials via vulnerable methods such as inserting them into an URL.

Many authorization methods use tokens and improper management of them also leads to weak security. Tokens might be stored in the browser where malicious code can access them, or expired and unsigned tokens might get accepted on the server side due to configuration errors or insufficient token checks. (Haro 2025, chapter 4.3);(OWASP 2023)

Code begins.

```
AUTH_PASSWORD_VALIDATORS = [  
  
    {'NAME': 'django.contrib.auth.password_validation.UserAttrib  
uteSimilarityValidator'},  
  
    {'NAME': 'django.contrib.auth.password_validation.MinimumLen  
gthValidator', 'OPTIONS': {'min_length': 15}},  
  
    {'NAME': 'django.contrib.auth.password_validation.CommonPass  
wordValidator'},
```

```
{'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator'}]
```

Code ends.

FIGURE 3. Configurations for strong password policies in Django settings. (Django REST framework s.a. b)

Django has many out of the box solutions for password validation (Figure 3.) and authentication. By asserting stronger passwords policies security can be improved drastically with little configuration.

Code begins.

```
class IsAuthenticated(BasePermission):
"""
Allows access only to authenticated users. Checks request
user which is identified by credentials against
authorization.
"""
def has_permission(self, request, view):
return bool(request.user and
request.user.is_authenticated)

class AuthExampleViewSet(viewsets.ReadOnlyModelViewSet):
"""
Checks authentication status against every request made to
the endpoint.
"""
permission_classes = [IsAuthenticated]
Code ends.
```

FIGURE 4. Minimal authentication solution in Django Rest Framework (Django REST framework s.a. a)

One method of authentication is using an API key, where the key needs to be included in every request made to the API. The inserted key is then checked against the user information and key stored in the server database for every request, granting authentication status that when valid, allows accessing the API. (Figure 4).

3.4 Broken Object Property Level Authorization

BOPLA is made up of two vulnerabilities, "Excessive Data Exposure" and "Mass assignment". These occur when a user is allowed to have unvalidated access to object properties using an API endpoint and giving too much data to the user in requests respectively. This allows a user to read and edit sensitive information that should not be sent to a user with responses.

Preventing this vulnerability requires validating and checking what properties the user should have access to and selecting the properties to be sent in to a client, removing any sensitive properties instead of using methods that automatically converts a whole data object into a sendable form. (Haro 2025, chapter 4.6);(OWASP 2023)

Code begins.

```
class BOPLAExampleSerializer(serializers.ModelSerializer):  
  
    class Meta:  
        model = User  
        fields = ["id", "username", "email", "role"]  
        read_only_fields = ["role"]
```

Code ends.

FIGURE 6. Selecting fields and read-only fields to be sent to a client in Django Rest Framework. (Django REST framework s.a. c)

Fields in Django Rest Framework can be explicitly specified in the object serializer that converts ORM models to JSON. Fields can be set as read only to make it so that they cannot be changed externally and are read-only. Sensitive fields and unnecessary fields can simply be omitted (Figure 6).

3.5 Broken function level authorization

Many applications and APIs have a concept of user groups or roles with different permissions and access levels. Checking these permissions is done by implementing role-based access controls, and this vulnerability occurs when a user manages to by-pass the access constraints of their role. This differs from

the previously mentioned BOLA in the way that in that vulnerability a user can access data from another user with the same role, while when function level authorization is broken a user can access functionalities such as admin-only operations.

An API could be built under the assumption that only legitimate users will access them, and because of that access controls would not be applied on every request. In that case anyone **who discovers** the admin endpoints could **elevate their privilege and gain access to** the admin functionalities of an API.

One way of mitigating this could be by having the access control checks in an API gateway, or by creating and applying a permission check to every request like in the previous Django Rest Framework example. A good strategy for token-based access controls could be to have user roles declared explicitly in access tokens and checking the roles on every request across all operations. (Haro 2025, chapter 4.7);(OWASP 2023)

3.6 Unrestricted access to sensitive business flows

Unrestricted access to sensitive business flows exposes APIs to various threats such as scalping, data manipulation, and scraping. This vulnerability is mitigated by reviewing API structure and identifying which business flows could be exposed and abused for malicious purposes. These vulnerabilities could be abused by automated bots or programs created by threat actors, therefore many protection mechanisms focus on slowing down attacks and involve tracking non-human behaviour.

Human behaviour detection could be implemented by using a CAPTCHA (Completely Automated Public Turing test to tell Computers and Humans Apart) or biometric solutions such as typing patterns and analysing the user flow to detect non-human patterns such as time spent in each screen or between each action, bots take milliseconds while humans typically take multiple seconds or even minutes. (Haro 2025, chapter 4.9);(OWASP 2023)

Code begins.

```

class ExampleHeaderPermission(BasePermission):
    allowed_agents = ["Mozilla", "Google Chrome", "Safari",
                     "Microsoft Edge"]
    allowed_header = "HTTP_USER_AGENT" # Django format

    def has_permission(self, request, view):
        user_agent = request.META.get(self.allowed_header, "")
        # Check if any allowed agent is present in the header
        for agent in self.allowed_agents:
            if agent in user_agent:
                return True
        return False

```

Code ends

FIGURE 7. A minimal permission to check for request user agents in Django Rest Framework. (Django REST framework s.a. a)

Another option could be to block IP addresses of Tor exit nodes and other well-known proxies and denying service to unexpected client devices such as headless browser that common bot software uses (Figure 7).

3.7 Unrestricted Resource Consumption

Satisfying API requests requires resources such as network bandwidth, CPU, memory, and storage. Required resources often cost money whether they are handled in an on-premises systems or cloud-based systems, additionally some resources are made available by service providers via API integrations and paid for per request. Unrestricted consumption of resources is a vulnerability that could cost a business a lot of money or stop legitimate customers from using their API/service.

Unrestricted resource consumption vulnerabilities can be exploited for brute-force attacks. An attacker could try to brute force other user's credentials by constantly spamming a login endpoint.

Resource consumption can be limited by implementing rate limits that deny clients who interact with a server too many times in a defined timeframe, limiting how often a client can execute a single operation, defining and enforcing maximum data sizes and lengths through validation, and by using solutions that

make it easy to limit resource usage, such as containers or serverless functions. (Haro 2025, chapter 5.1);(OWASP 2023)

Code begins.

```
REST_FRAMEWORK = {
    'DEFAULT_THROTTLE_CLASSES': [
        'rest_framework.throttling.AnonRateThrottle',
        'rest_framework.throttling.UserRateThrottle'
    ],
    'DEFAULT_THROTTLE_RATES': {
        'anon': '5/minute',
        'user': '1000/hour'
    }
}
```

Code ends.

FIGURE 8. Enabling IP-based throttling in Django Rest Framework. Authenticated requests are permitted up to 1000 requests per hour, while non-authenticated requests are limited to 5 per minute. Custom throttle rules such as different rates for different customer roles could be applied. (Django REST framework s.a. d)

This kind of throttling policy helps to protect the system from abuse and accidental overloads, but as IP addresses can be easily spoofed, it is not a proper defence mechanism against attacks such as DoS (Figure 8). (Django REST framework s.a. d)

DoS or DDoS are attacks where a malicious entity attempts to disrupt normal traffic of a targeted server, service, or network by overwhelming the target or its surrounding infrastructure with a flood of Internet traffic. Attacks are carried out from a network or botnet that consists of computer and other devices such as IoT devices which have been infected with malware, allowing them to be controlled remotely by an attacker. DDoS attacks can be very sophisticated and even properly secured services might struggle against them. These attacks are particularly effective against systems without proper resource consumption control. (Cloudflare s.a. b)

3.8 Source Code Analysis Tools

SAST tools can help analyse source code or compiled versions of code to find security flaws. The code analysed is static, which means that it is analysed without running the code. These kinds of tools often work for many programming languages and can be integrated into an IDE or a CI/CD pipeline, whether it is cloud or server based.

Advanced SAST tools can be used to configure coding rules to ensure all code adheres to security standards, is compliant, and is unified in quality. A previously mentioned risk is vulnerabilities or attacks through the supply chain of open-source libraries, SCA tools scan these libraries and analyse these risks to prevent them. Because many vulnerabilities also arise from misconfiguration, these tools also scan for configuration files and content that should not be in the codebase such as secrets.

There are both commercial and non-commercial SAST tools and they are an easy method to improve code security and apply code rules on a large scale. (OWASP s.a. c.);(GitHub 2024.)

4 RISK MANAGEMENT IN WEB APPLICATIONS

Cyberattacks occur constantly and have a very wide range of actors, targets, and attack methods. To effectively mitigate these threats, security should be considered at all levels of an application, including code, infrastructure, and system architecture. In addition to technical controls, security should also be embedded into the organizational structure of a company through clear access policies, regular vulnerability scanning, and internal audits that monitor both systems and internal actions.

Due to the nature of maintaining and developing services, all these aspects should be considered consistently for every new service created and functionality developed. Many mitigation methods can be simple to implement, and consistent adherence to good security practices helps reduce vulnerabilities while building trust in a service, particularly when that service is offered as a commercial product.

Defending against cyberattacks is a matter of risk management because an assumption should be made that eventually some incident will happen, and the impact of that eventuality should be prepared for and made as small as possible.

Mitigation is about managing cybersecurity risks and preparing for the worst, while a successful attack can temporarily disrupt a service, effective mitigation ensures that damage is contained. By minimizing data loss and service downtime, both the integrity of the system and the trust of its users can be preserved. (Lindemulder, Kosinski & Jonker 2026.)

4.1 API Governance

Governance plays a critical role in cybersecurity and vulnerability mitigation. When applied on an organizational level, it enforces consistent policies and standards into everyday operations, ensuring that security weaknesses are

identified early, addressed systematically, and prevented from escalating into widespread incidents. (Postman s.a.);(Dynowski & Dulak 2025, chapter 1.)

4.1.1 Regulation

Governance entails adherence to security standards, laws, compliance requirements, and regulations. For high profile organizations or those operating withing regulated industries such as financial or medical operations, compliance is stricter. All applicable requirements must be followed throughout the entire software lifecycle, from design and development to deployment and operation.

A European company may be subject to both national legislation and European Union regulations. This creates its own set of obligations that software systems must satisfy. Failure to comply with these requirements can result in significant financial penalties, legal consequences, and reputational damage, placing the organization in a vulnerable position (Haro 2025, chapter 10.);(ENISA s.a.);(European Commission 2024.)

4.1.2 Assessment

To determine if an organization is compliant with regulations and law, external audits are conducted. These audits are performed by regulatory authorities or independent security organizations with the authority to verify compliance and assess the security posture of an organization. Their role is to ensure that required controls are in place and effectively enforced.

In contrast, internal audits are performed by teams within the organization and assess if the software managed by the organization is secure. These audits assess whether internal policies, processes, and technical controls are being followed in practice. Internal audits often cover multiple organizational levels and help identify weaknesses in software, processes, or access management before they are exploited.

Security standards such as ISO27001 provide structured guidance for implementing an Information Security Management System (ISMS). By meeting the requirements of such standards, organizations can systematically protect sensitive data, manage risk, and reduce exposure to attacks. By having certifications of such standards, an organization can show that they meet security standards. (IBM s.a. a.);(ISO-CC s.a.)

4.1.3 Management

As an organization grows, effective management becomes increasingly important. Governance must extend beyond the software itself to include access control, user permissions, and to educating individuals who interact with the software. Access to critical components and administrative functionality should be restricted according to the principle of least privilege, ensuring that users only have the permissions necessary to perform their tasks. (OWASP s.a. b.)

Individuals with higher levels of access are more likely to be targeted by attackers, as compromising privileged accounts provide greater opportunities for abuse. As such, employees in privileged roles should be educated to understand that they are potential targets. Security awareness training helps employees recognize threats and take proactive steps to avoid becoming easy entry points for attackers. By managing both technical access and human behaviour, organizations can significantly reduce the likelihood and impact of successful attacks. (IBM s.a.)

Third-party dependencies used in a service, and especially open-source dependencies should also be managed and audited. The dependencies have their own dependencies, creating a software supply chain where someone could plant backdoors or malicious code into a dependency far into the chain without being detected. Securing the software supply chain is essential for maintaining compliance with industry regulations and standards, and mitigates risk associated with software vulnerabilities. (Jfrog s.a.)

4.2 Architecture

Many different approaches can be taken to improve security through software architecture, as it is the backbone of any secure system. Requirements vary depending on the kind of system and use cases, and whether the connections are coming from outside to inside. In this chapter a few common ways to create secure architecture are explored. (Haro 2025.)

4.2.1 API gateway

In the context of this work an API gateway is API management-solution software, these solutions offer many features that simplify API governance, including authentication, authorization checks, monitoring, inventory management, schema validation, and rate limiting.

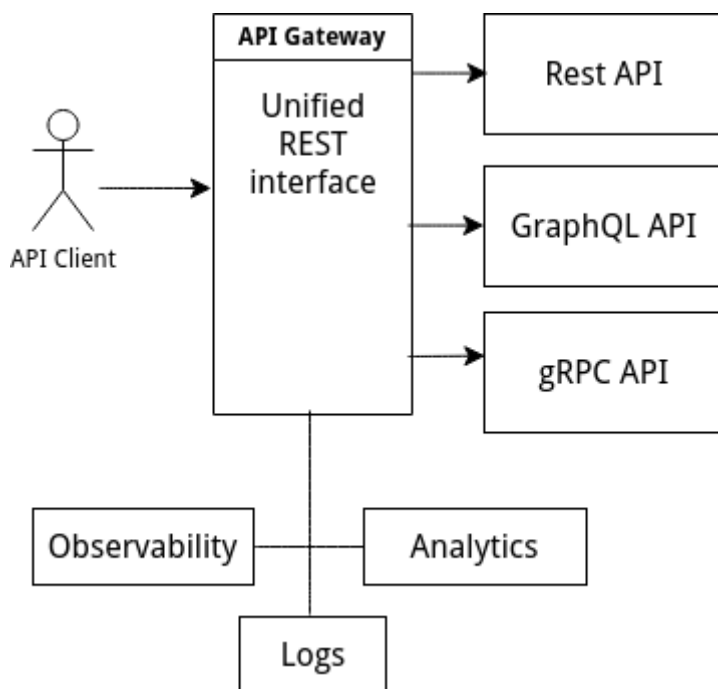


FIGURE 10. An API gateway working as a single point of entry. Logging, Observability, and Analytics capabilities can help manage misbehaving API clients and follow API usage patterns.

With an API gateway layer applied between a client and other services, standardized treatment can be applied to every incoming request while

redirecting them to corresponding services in a safe manner, without needing to copy the same functionality in other services connected to the gateway. It can also work as a protocol and API style translation layer, offering a single type of interface such as REST for incoming traffic while redirecting requests to other APIs that could be of different protocol such as GraphQL or gRPC (Figure 10). (Cloudflare s.a. c.);(Haro 2025.)

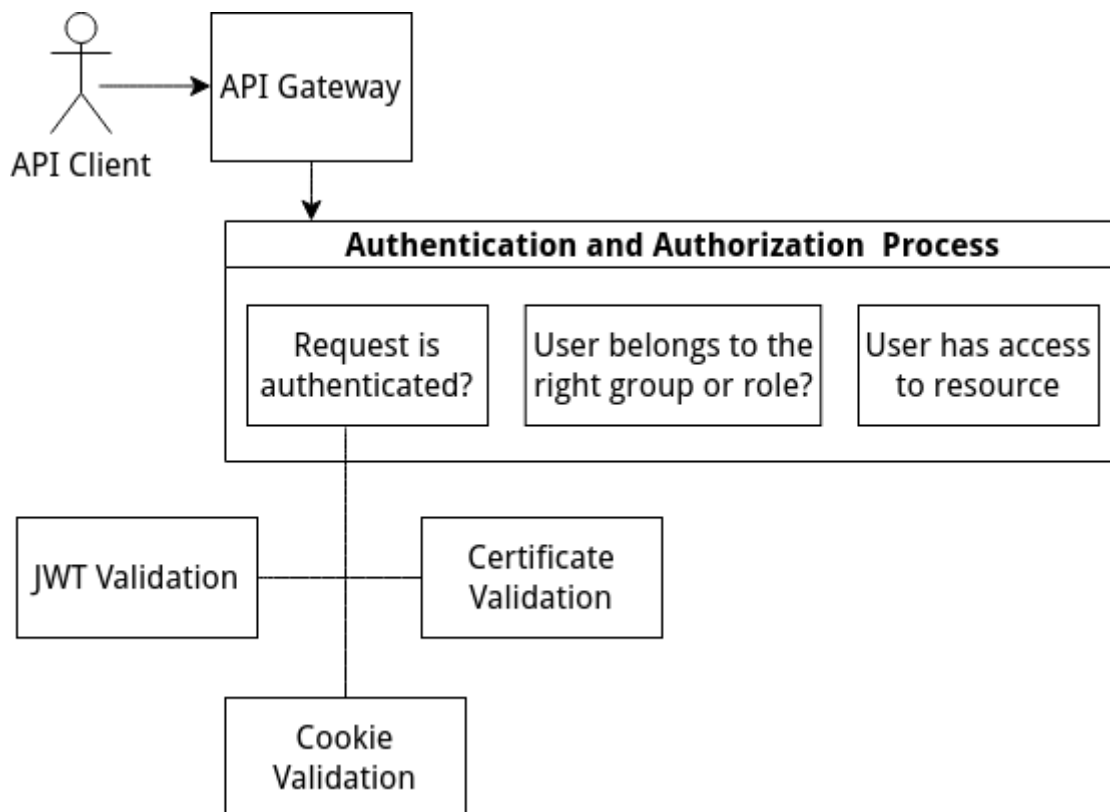


FIGURE 11. Applying consistent access control using an API gateway. Applying rules in a single place mitigates vulnerabilities that are caused by broken authentication and human error.

API gateways can help to apply other many kinds of secure configurations to requests and responses throughout the whole application without duplicate implementations. API gateways can validate requests and responses against schemas, enforcing design constraints on request and response bodies, URL parameters, and headers. Headers such as Access-Control-Allow-Origin for CORS, Strict-Transport-Security for HSTS, and Content-Security-Policy for content security policy can be injected to all responses passing through the API gateway (Figure 11). (Cloudflare s.a. c.);(Haro 2025.)

4.2.2 Network Topology

A network can be segmented into multiple subnets, each segment representing a small network with various levels of access. Some might have access to the internet while others are isolated to a private network. Network topology refers to the structure of a network and is an essential part of securing services.

Through this kind of arrangement only a part of the network needs to be exposed to the internet, limiting the attack surface of a network. This section of the network is known as a perimeter network, and it may include elements such as an API gateway and special fortified server called a bastion server, which allows only inbound traffic through a single port such as 22 for SSH connections for a small number of trusted IP addresses.

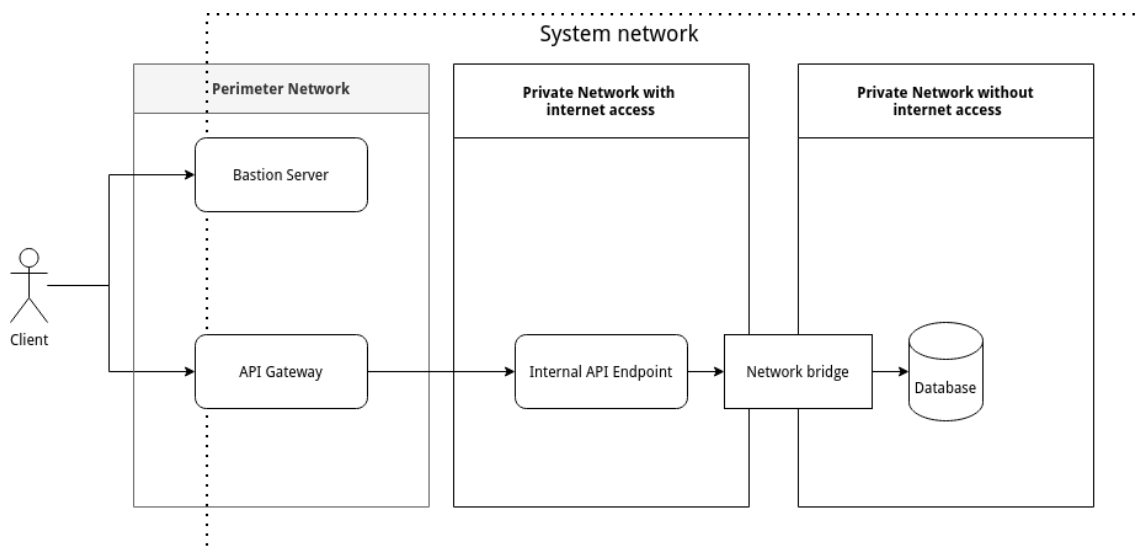


FIGURE 12. Through architecture decisions, it is possible to create a situation where an API is not accessible unless the endpoints are declared in the API gateway, making sure only documented, tested, and secure endpoints are exposed to the internet while internal endpoints are accessible only to internal services.

Network segmentation is a very good way of isolating services that have sensitive data and operations, as there is no way to connect directly to a different network even if an attacker could access some part of the system through the internet (Figure 12).(Cloudflare s.a. d.);(Haro 2025.)

5 INCIDENT RESPONSE

In software development, events such as errors in production environments and attacks are called incidents. Detecting and reacting to incidents is crucial in maintaining software and minimizing the impact of cyberattacks and other threats, even physical threats such as fire, volcanic eruptions, and war.

5.1 Observability

For an organization to be able to respond to a threat, that threat must first be detected. Detection can be achieved with the practice of API observability. Traces, logs, and metrics are constantly collected from API operations to identify unordinary user behaviour, bugs, and errors. Observability answers to the questions of when, where and who used a software system. These statistics are also useful when debugging services.

Logs are records of specific events, when something happens in a system, such as when an error is raised, a log of that event is issued. However, with just logs, it is difficult to find where exactly the error occurred. Traces ease this problem by recording the processing steps taken to process a request throughout a system.

Metrics are a measure of events **that happen in** an application, they capture details on resource and systems behaviour. Metrics could be things such as error rate, latency, or requests per second. These previously mentioned concepts are the backbone of observability and enable the monitoring of a system.

Monitoring is the practice of using this captured data to create alerts to be triggered when certain metrics cross certain threshold. **Monitoring**, metrics, traces, and logs all complement each other and create an observable software system. (Haro 2025.)

5.2 Response

For responses to be swift, they need to be planned beforehand. Organizations should have an incident response plan that documents what kind of action should be taken in the event of various kinds of scenarios such as certain attacks, catastrophic events, and long-lasting service disruptions. The plan specifies who is in charge when an incident occurs, what are the steps to be taken, and who perform the steps.

Planning for incidents justifies making arrangements such as data backups, redundant equipment, and training people on how they should be reacting. An incident response plan minimizes operational disruptions, financial losses, and reputational damage.

Many established organizations have an incident response team comprised of people trained and authorized to handle a security incident, also called the Computer Security Incident Response Team. While these teams can include both internal and external third-party security providers, CSIRT teams are responsible for all steps taken when responding to an incident, these steps according to the SANS institute response framework consists of six phases.

Preparation, that focuses on understanding the security environment, its network, and the people using it. Policies and protocols for various incident types are created, training and simulations for staff members are conducted.

Identification, where the goal is to identify how serious an incident is, anomalies are detected through monitoring and reports, logs, alerts, and data are analysed, and the scope of the threat is identified.

Containment, where the goal is to stop malicious activity from continuing to operate within an organization's network or system, minimizing damage. Spread of the threat is limited by isolating affected systems and containment strategies are implemented.

Eradication, that focuses on removing the harmful program or threat actor from the network and system, then reverting actions or changes they made. Malicious

elements are removed and vulnerabilities patched while performing root cause analysis to prevent recurrence.

Recovery, that focuses on getting information systems back up and running safely. Affected systems, data, files, and configurations are restored to normal operation using backups and monitoring is performed for signs of residual threats or vulnerabilities.

Lessons Learned, where post-incident reviews are conducted to identify areas of improvement and feedback and review process to improve future responses is undertaken. Countermeasures are developed and policies, procedures, communications plans, and technologies are updated based on insights gained. (Murdoch 2026, chapter 2.);(Sans 2026.)

Responding to an incident well and keeping it from happening again is one of the key goals of incident response, that protects both and organization and its customers from harm.

6 FUTURE DEVELOPMENT AND DISCUSSION

This thesis examined cybersecurity in API development by combining technical implementation practices with organizational and architectural considerations. The focus was placed on publicly exposed, internet-facing APIs, with practical examples presented using Django Rest Framework.

6.1 Discussion

Through research it was discovered that API security is not achieved through a single control or technology, but instead through the layered and consistent application of security principles across development, deployment, and organizational processes.

An important realization is that many of the most severe API vulnerabilities arise not from complex exploits, but from simple human error and negligence. Vulnerabilities such as BOLA and BFLA happen often due to assumptions about trusted users or incomplete permission checks. Shadow APIs, misconfigurations, and outdated endpoints are governance failures by the organization and people responsible for them rather than purely technical failures. In some cases, security might not be considered much at all because there might not be a business case for using the time and effort to make an application more secure.

The examples shown in this work demonstrate that modern frameworks such as Django Rest Framework already provide a wide array of effective mechanism to mitigate security risks, but their effectiveness depends on correct and systematic usage of development technologies and tools. This also shows the importance of secure development practices during the whole development process and a consistent risk management process.

Architectural components such as API gateways and network segmentation provide multiple lines of defence against threat actors and enforce rules upon network connections in centralized entry points and mitigate risks created by

human error, however these should be considered only as a part of the total API security approach of a system. Implementation level checks and security methods in code are equally as important in creating a secure system.

Having comprehensive logs, traces, and metrics of services is an important part in preparing for an incident and provides ways to audit, debug, and monitor a system. Through detecting anomalies using these statistics organizations can respond to an incident effectively. Being prepared for incidents and having a plan for them minimizes negative effect on the users of a system and on the reputation of its owners.

6.2 Future development

This thesis primarily focuses on practical approaches to improving API security. While each of the presented topics is valuable on its own, a comprehensive treatment of each would require a more in-depth analysis that is out of the scope of this work.

Several built-in defensive mechanisms provided by Django and Django Rest Framework were not examined in detail. These include Django's built-in protection against CSRF through CSRF tokens, as well as its mitigation of SQL injection vulnerabilities via the Django ORM. CSRF and SQL injection attacks could warrant a more throughout explanation.

In addition, this work does not provide implementation-level research into the detection of non-human behaviour or abusive usage patterns related to API keys. Investigating these approaches in more detail would be a natural extension of this thesis and could serve as the foundation for implementing more advanced security methods.

REFERENCES

China, C & Goodwin, M. s.a. What is API security? Available at: <https://www.ibm.com/think/topics/api-security>. Accessed 24 March 2026.

Cloudflare 2024. 2024 API Security & Management Report. Available at: <https://www.cloudflare.com/2024-api-security-management-report/>. Accessed: 07 February 2026.

Cloudflare s.a c. What is an API Gateway?. Available at: <https://www.cloudflare.com/learning/security/api/what-is-an-api-gateway/>. Accessed: 25 March 2026.

Cloudflare s.a d. What is the network perimeter? Available at: <https://www.cloudflare.com/learning/access-management/what-is-the-network-perimeter/>. Accessed: 25 March 2026.network-perimeter/. Accessed 25 March 2026.

Cloudflare s.a. a. What is an API?. <https://www.cloudflare.com/learning/security/api/what-is-an-api/>. Available at: Accessed: 08 February 2026. /. Accessed: 08 February 2026.

Cloudflare s.a. b. What is a DDoS attack?. Available at: <https://www.cloudflare.com/learning/ddos/what-is-a-ddos-attack/>. Accessed: 20 March 2026.

Django REST framework s.a. a. Permissions. Available at: <https://www.django-rest-framework.org/api-guide/permissions/>. Accessed: 29 March 2026

Django REST framework s.a. b. Validators. Available at: <https://www.django-rest-framework.org/api-guide/validators/>. Accessed 29 March 2026

Django REST framework s.a. c. Serializers. Available at: <https://www.django-rest-framework.org/api-guide/serializers/>. Accessed 29 March 2026.

Django REST framework s.a. d. Throttling. Available at: <https://www.django-rest-framework.org/api-guide/throttling/>. Accessed 29 March 2026.

Dynowski, L & Dulak, M 2025. Learning API Styles. O'Reilly Media, Inc. Accessed: 10 February 2026. Available at: <https://learning.oreilly.com/library/view/learning-api-styles/9781098153984/>. Access required.

ENISA s.a. Cybersecurity of Critical Sectors. Available at: <https://www.enisa.europa.eu/topics/cybersecurity-of-critical-sectors>. Accessed 10 March 2026.

European Commission 2024. The Cyber Resilience Act – Summary of the legislative text. Available at: <https://digital-strategy.ec.europa.eu/en/policies/cra-summary>. Accessed 10 March 2026.

Gallagher, N. & Goodwin, M. 01 October 2025. What is API management? Available at: <https://www.ibm.com/think/topics/api-management>. Accessed: 03 March 2026.

GitHub 2024. Available at: <https://github.com/resources/articles/what-is-software-composition-analysis>. Accessed 17 March 2026.

Haro, J. 2025. Secure APIs. Manning Publications. Accessed: 20 February 2026. Available at: learning.oreilly.com/library/view/secure-apis/9781633436633/. Access required.

IBM s.a. a. What is ISO 27001? Available at: <https://www.ibm.com/products/cloud/compliance/iso-27001>. Accessed 13 March 2026.

IBM s.a. b. What is cyber risk management?. Available at: <https://www.ibm.com/think/topics/cyber-risk-management>. Accessed: 10 March 2026.

ISO-CC s.a. The Role of Internal vs. External Audits in ISO 27001 Compliance. Available at: <https://iso-cc.com/the-role-of-internal-vs-external-audits-in-iso-27001-compliance/>. Accessed 12 March 2026.

Jackson, G & Goodwin, M .s.a. What is API governance? Available at: <https://www.ibm.com/think/topics/api-governance>. Accessed 03 March 2026.

Jfrog s.a. What is Software Supply Chain Security. Available at: <https://jfrog.com/learn/software-supply-chain/>. Accessed 10 March 2026.

Lindemulder, G., Kosinski, M & Jonker A. 2026. What is cybersecurity?. Available at <https://www.ibm.com/think/topics/cybersecurity>. Accessed 14 March 2026.

Microsoft 2025. Use API gateways in microservices. Available at: <https://learn.microsoft.com/en-us/azure/architecture/microservices/design/gateway>. Accessed: 29 March 2026

Mozilla 2025. HTTP. Available at: <https://developer.mozilla.org/en-US/docs/Glossary/HTTP>. Accessed: 29 March 2026

Murdoch, D. 2026. Blue Team Handbook: Incident Response. O'Reilly Media, Inc. Accessed: 04 April 2026. Available at: <https://learning.oreilly.com/library/view/blue-team-handbook/9798341649767/>. Access required.

OWASP 2023. OWASP API Security Top 10. Available at: <https://owasp.org/API-Security/editions/2023/en/0x00-header/>. Accessed: 29 February 2026.

OWASP 2026. About the OWASP Foundation. Available at: <https://owasp.org/about/>. Accessed: 29 February 2026.

OWASP s.a. a. OWASP API Security Project. Available at:

<https://owasp.org/www-project-api-security/>. Accessed 27 March 2026.

OWASP s.a. b. Least Privilege Principle. Available at: [https://owasp.org/www-](https://owasp.org/www-community/controls/Least_Privilege_Principle)

[community/controls/Least_Privilege_Principle](https://owasp.org/www-community/controls/Least_Privilege_Principle). Accessed: 20 March 2026.

OWASP s.a. c. Source Code Analysis Tools. Available at:

https://owasp.org/www-community/Source_Code_Analysis_Tools. Accessed

March 21 2026.

Pedro, B. 2024. Building an API Product. Packt Publishing. Accessed: 15

February 2026. Available at: [learning.oreilly.com/library/view/building-an-](https://learning.oreilly.com/library/view/building-an-api/9781837630448/)

[api/9781837630448/](https://learning.oreilly.com/library/view/building-an-api/9781837630448/). Access required.

Postman s.a. What is API governance?. Available at:

<https://www.postman.com/api-platform/api-governance/>. Accessed: 20 March

2026.

Sandoval, K. 18 December 2025. 9 Types of API Monetization Models.

Available at: <https://nordicapis.com/9-types-of-api-monetization-models/>.

Accessed 28 February 2026.

Sans 2026. Incident Response. Available at: [https://www.sans.org/security-](https://www.sans.org/security-resources/glossary-of-terms/incident-response)

[resources/glossary-of-terms/incident-response](https://www.sans.org/security-resources/glossary-of-terms/incident-response). Accessed: 05 April 2026